Project Report

Theme: Fine-Tuning LLMs for Children's Story Generation

This NLP project focuses on domain adaptation by fine-tuning models like Google's Gemma 4 (E2B-IT) and GPT-2 Small to generate educational and engaging children's stories. The pipeline includes model quantization, merging, and deployment using GGUF for efficient local inference, with baseline comparisons against Gemma 4 E4B-IT and GPT-2 XL.

Group Members:

Benhammadi Lokmane

Khedim Youcef

Group 1
Speciality: IASD

GitHub Repository (Source Code):
https://github.com/khedimyoucef/NLP-MINI-PROJECT

May 4, 2026

Contents

List of Figures

1. Introduction and Problem Statement

1.1. The Challenge: Domain-Specific Text Generation

Large Language Models (LLMs) are incredibly capable zero-shot generalists. However, when tasked with creating highly specific content—such as educational stories tailored to exact age groups—they often drift into generic language, hallucinate complex vocabulary, or fail to adhere to structural constraints.

Two primary challenges define this project:

- **Style Adaptation:** Adjusting the model's tone to be engaging, safe, and comprehensible for young children, categorized strictly by age levels.
- **Resource Constraints:** Fine-tuning billions of parameters requires substantial compute memory. Developing a pipeline capable of running on accessible hardware (like Kaggle's dual T4 GPUs) is critical.

1.2. Our Approach: Multi-Model Fine-Tuning and Edge Deployment

We fine-tuned Google's `gemma-4-E2B-it` and `gpt2-small` models on a curated children's story dataset. To bypass hardware limitations for the larger Gemma model, we utilized 4-bit Quantized Low-Rank Adaptation (QLoRA). Once trained, the adapters were merged back into the base model and converted into a highly optimized `.gguf` format, enabling fast, local inference on edge devices via `llama.cpp`. For comparison, we also included two base models: `gpt2-xl` and `gemma-4-E4B-it`.

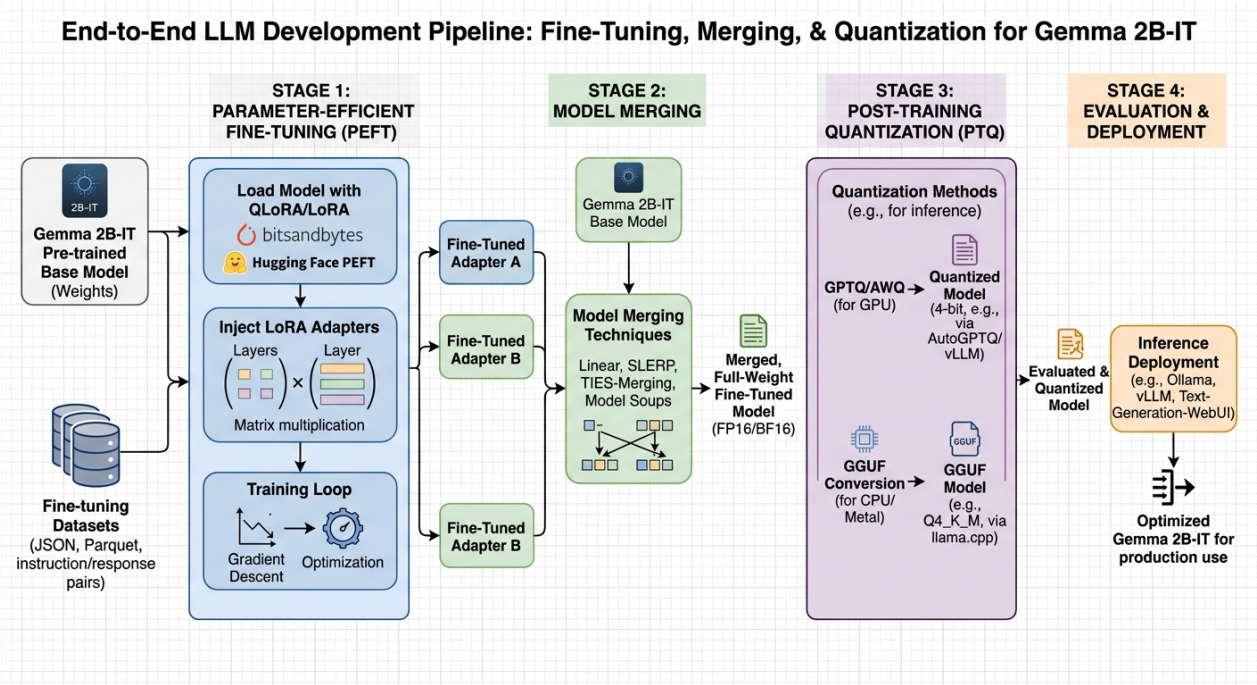


Figure 1: End-to-end LLM fine-tuning, merging, and quantization pipeline.

2. Background: Parameter-Efficient Fine-Tuning

2.1. The Memory Bottleneck

Training an LLM requires memory for model weights, gradients, optimizer states, and activations. A 2-billion parameter model in 16-bit precision requires roughly 4GB just to hold the weights, but over 16GB to train fully. This makes full fine-tuning inaccessible for consumer GPUs.

2.2. Quantization and QLoRA

To solve this, we employ **QLoRA** for the Gemma model. The base model is loaded in 4-bit precision (NF4), drastically reducing the memory footprint. Instead of updating the original weights, QLoRA injects small, trainable "adapter" matrices into the model's layers. During training, only these adapter weights are updated while the base model remains frozen.

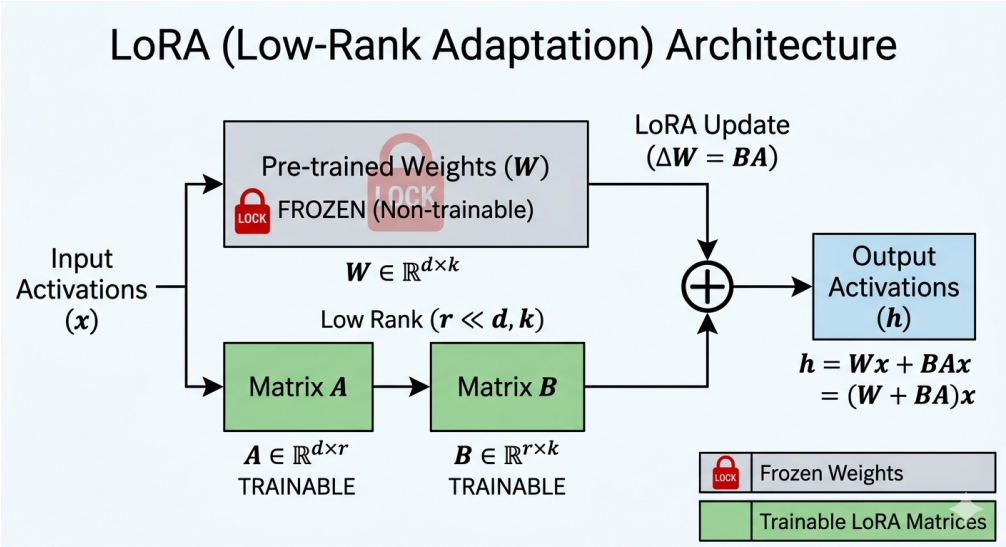


Figure 2: LoRA architecture: small rank decomposition matrices are trained while the pre-trained weights remain frozen.

3. Model Architectures and Selection

For this project, we employed a mix of fine-tuned and base models to thoroughly evaluate the impact of domain adaptation against out-of-the-box generalization.

3.1. Gemma 4 E2B-IT (Fine-Tuned)

We selected the `gemma-4-E2B-it` (Instruction Tuned) variant, featuring 2.3 billion parameters. It offers a high capacity-to-size ratio, fits comfortably on a single 16GB GPU, and is pre-aligned to follow instructions. Its massive 256,000-token vocabulary gives it exceptional structural generation capacity.

3.2. GPT-2 Small (Fine-Tuned)

As a lightweight alternative to Gemma, we fully fine-tuned `gpt2-small` (124M parameters). Being much smaller, it allowed us to experiment with full fine-tuning without the need for QLoRA. It serves as our baseline for domain-adapted text generation with limited capacity.

3.3. Base Models for Comparison

To contrast our fine-tuned models with zero-shot generalists, we included two base models in our evaluation pipeline:

- **GPT-2 XL (1.5B parameters):** A larger version of GPT-2 to observe how scaling parameters compensates for lack of domain-specific fine-tuning.
- **Gemma 4 E4B-IT (~4B parameters):** A significantly larger variant of the Gemma family. We quantized it to GGUF format to run locally and use it as a high-end baseline.

4. Dataset

4.1. Source and Formatting

We utilized the `ajibawa-2023/Children-Stories-Collection` dataset from Hugging Face. The dataset consists of story prompts and corresponding stories suitable for children.

To teach the model to adapt its vocabulary, we prefixed each prompt with an age-level constraint. The training samples were formatted into Gemma's native instruction format:

```
<start_of_turn>user
Level: Age5-6 - Short sentences.
Write a story about a brave little toaster.<end_of_turn>
<start_of_turn>model
Once upon a time, there was a little silver toaster. He lived in a warm kitchen...<
    end_of_turn>
```

4.2. Dataset Statistics

We subsampled 10,000 stories for this fine-tuning pass, splitting 95% for training and 5% for evaluation.

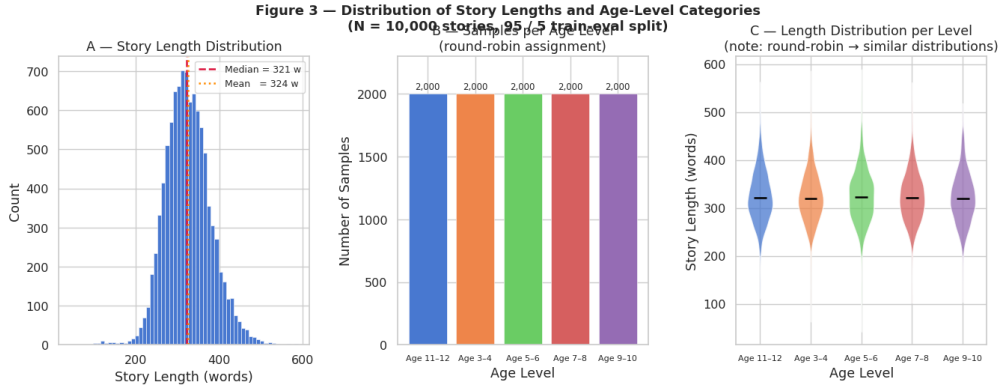


Figure 3: Distribution of story lengths and age-level categories in the training sample.

5. Training Methodology

5.1. Configuration and Hardware

Training was orchestrated using the Hugging Face `Trainer` API on a Kaggle environment equipped with dual NVIDIA T4 GPUs.

Key training parameters for Gemma:

Training Arguments

```
training_args = TrainingArguments(
    per_device_train_batch_size=1,
    gradient_accumulation_steps=16,
    optim='paged_adamw_8bit',
    learning_rate=2e-4,
    fp16=True,
    gradient_checkpointing=True,
    eval_strategy='no' # Disabled to prevent OOM
)
```

5.2. Overcoming Memory Constraints

Due to Gemma 4's massive 256K token vocabulary, calculating cross-entropy loss over large sequences creates a massive memory spike. We mitigated this by:

1. Enforcing `batch_size = 1` with high gradient accumulation (16 steps) to simulate a batch size of 16.
2. Enabling `gradient_checkpointing` to trade computation time for VRAM savings.
3. Setting `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` to prevent CUDA memory fragmentation.

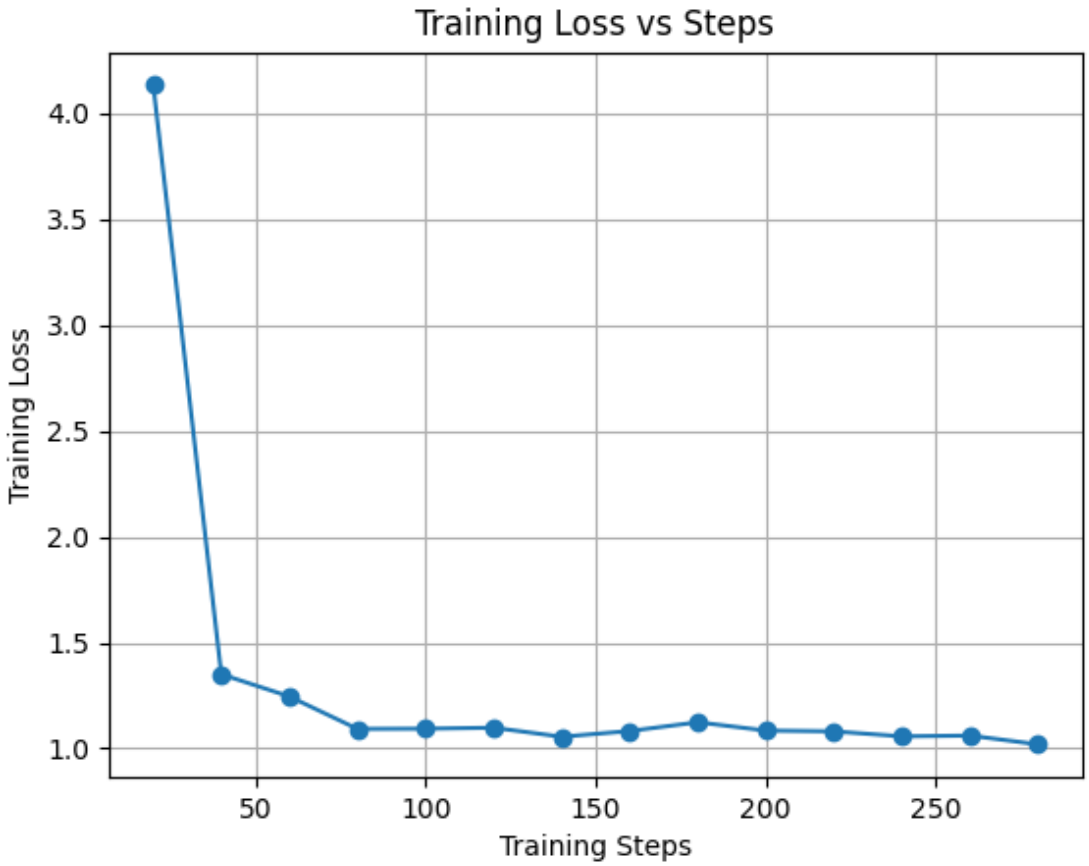


Figure 4: Training loss curve over the 1-epoch fine-tuning run. The loss fluctuates over the steps but maintains an overall downward trend, indicating effective learning and adaptation to the domain.

6. Model Export and Deployment

6.1. Merging the Adapters and Quantization

For the Gemma model, QLoRA results in standalone adapter weights. We reloaded the base model in FP16 precision on the CPU, injected the LoRA weights, and invoked `merge_and_unload()` to fuse them permanently.

To deploy the model locally, we converted the merged model into the GGUF standard using `llama.cpp`. The FP16 model was further quantized down to `Q4_K_M` (4-bit, medium quantization). This reduced the model's disk footprint to approximately 1.5 GB, allowing it to run entirely on consumer CPU RAM.

Another problem we faced was the management of Kaggle's limited RAM and storage resources, so we had to implement a pipeline that constantly monitored and cleaned up unnecessary files after each stage.

```
============================================================
STAGE 1: TRAIN
============================================================
✅ Final adapter found on HF Hub — skipping training.
   Downloading adapter to /kaggle/working/lora_adapter...
Loading widget...
Loading widget...
   Adapter ready locally.

✅ STAGE 1 COMPLETE
```

```
============================================================
STAGE 2: MERGE TO DISK
============================================================

  [1/4] Clearing HF cache to make room on disk...
  💾 Disk free [after cache wipe]: 20.8 GB

  [2/4] Loading base model (FP16)...
Loading widget...
 [transformers] `torch_dtype` is deprecated! Use `dtype` instead!
Loading widget...
Loading widget...
Loading widget...
  Unwrapped 232 Gemma4ClippableLinear modules
  [3/4] Merging LoRA weights...
  [4/4] Saving merged model to /kaggle/working/gemma_merged...
Loading widget...

✅ STAGE 2 COMPLETE
   Disk used: 10.4 GB
   RAM free: 15.0 GB
```

Figure 5: The llama.cpp conversion and quantization process log (part 1).

```
============================================================
STAGE 3: CONVERT & QUANTIZE
============================================================
  💾 Disk free [before GGUF conversion]: 10.5 GB

  Building llama.cpp (one-time, ~5 min)...
Cloning into '/kaggle/working/llama.cpp'...
-- The C compiler identification is GNU 11.4.0
-- The CXX compiler identification is GNU 11.4.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Found Git: /usr/bin/git (found version "2.34.1")
-- The ASM compiler identification is GNU
-- Found assembler: /usr/bin/cc
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
CMAKE_BUILD_TYPE=Release
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
-- Found Threads: TRUE
-- Warning: ccache not found - consider installing it for faster compil
-- CMAKE_SYSTEM_PROCESSOR: x86_64
-- GGML_SYSTEM_ARCH: x86
-- Including CPU backend
```

```
[ 97%] Building CXX object tests/CMakeFiles/test-chat.dir/get-model.cpp.o
[ 98%] Building CXX object tests/CMakeFiles/test-gguf-model-data.dir/test-gguf-model-data.cp
[ 98%] Linking CXX executable ../bin/test-gguf-model-data
[ 98%] Built target test-gguf-model-data
[ 98%] Building CXX object tests/CMakeFiles/test-quant-type-selection.dir/test-quant-type-se
[ 98%] Building CXX object tests/CMakeFiles/test-quant-type-selection.dir/get-model.cpp.o
[ 99%] Linking CXX executable ../bin/test-quant-type-selection
[ 99%] Built target test-quant-type-selection
[ 99%] Building CXX object tests/CMakeFiles/export-graph-ops.dir/export-graph-ops.cpp.o
[ 99%] Linking CXX executable ../bin/export-graph-ops
[ 99%] Built target export-graph-ops
[100%] Linking CXX executable ../../bin/llama-server
[100%] Built target llama-server
[100%] Linking CXX executable ../bin/test-chat
[100%] Built target test-chat
  ✅ llama.cpp built.

  Converting /kaggle/working/gemma_merged → FP16 GGUF...
```

Figure 6: The llama.cpp conversion and quantization process log (part 2).

```
  ✅ FP16 GGUF: 9.27 GB
  Freeing /dev/shm (~9 GB RAM)...
  💾 Disk free [after /dev/shm free]: 11.2 GB

  Quantizing to Q4 K M...
```

```
main: quantize time = 193147.70 ms
main:    total time = 193147.70 ms
  ✅ Q4_K_M GGUF: 3.42 GB
  💾 Disk free [end of Stage 3]: 17.0 GB

✅ STAGE 3 COMPLETE
```

```
 ============================================================
 STAGE 4: UPLOAD
 ============================================================
   Uploading 3.42 GB to khedim/NLP-MINI-PROJECT...
Loading widget...
Loading widget...
   ✅ Upload complete.
   💾 Disk free [final]: 20.4 GB

 ============================================================
 🎉 ALL DONE!
    GGUF : https://huggingface.co/khedim/NLP-MINI-PROJECT
    LoRA : https://huggingface.co/khedim/NLP-MINI-PROJECT-adapter
 ============================================================
```

Figure 7: The llama.cpp conversion and quantization process log (part 3).

The final GGUF file was tested on a local machine using `llama.cpp` to verify that the generation quality was preserved.

6.2. Hugging Face Repositories

The resulting models were uploaded to Hugging Face for persistence:

- **Fine-Tuned Gemma (GGUF):**
  https://huggingface.co/khedim/NLP-MINI-PROJECT/tree/main/gemma-e2b
- **Fine-Tuned GPT-2 Small:**
  https://huggingface.co/khedim/NLP-MINI-PROJECT/tree/main
- **Gemma LoRA Adapter Weights:**
  https://huggingface.co/khedim/NLP-MINI-PROJECT-adapter/tree/main

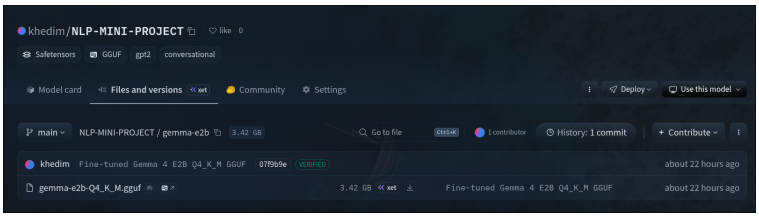


Figure 8: Gemma model repository on Hugging Face.

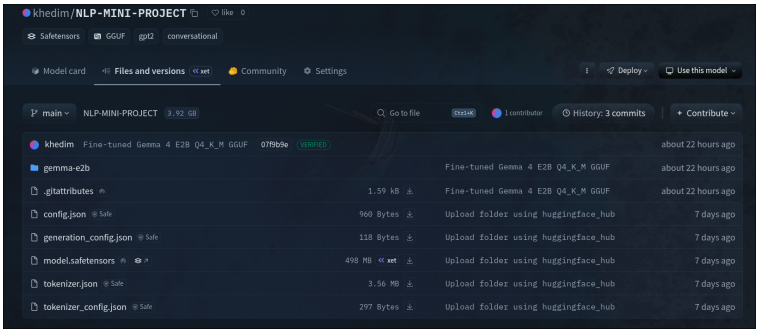


Figure 9: GPT-2 Small model repository on Hugging Face.

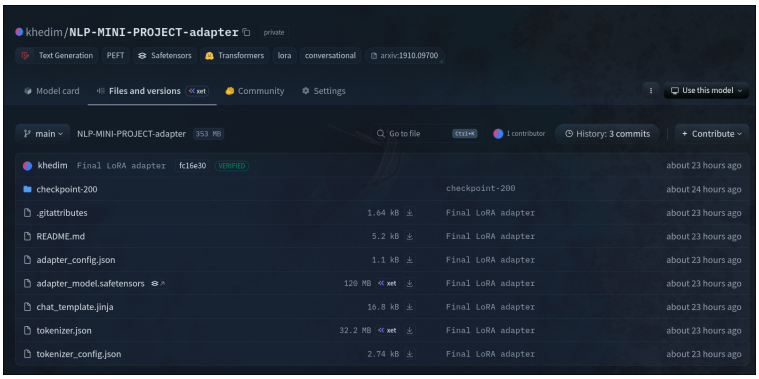


Figure 10: Adapter weights repository on Hugging Face.

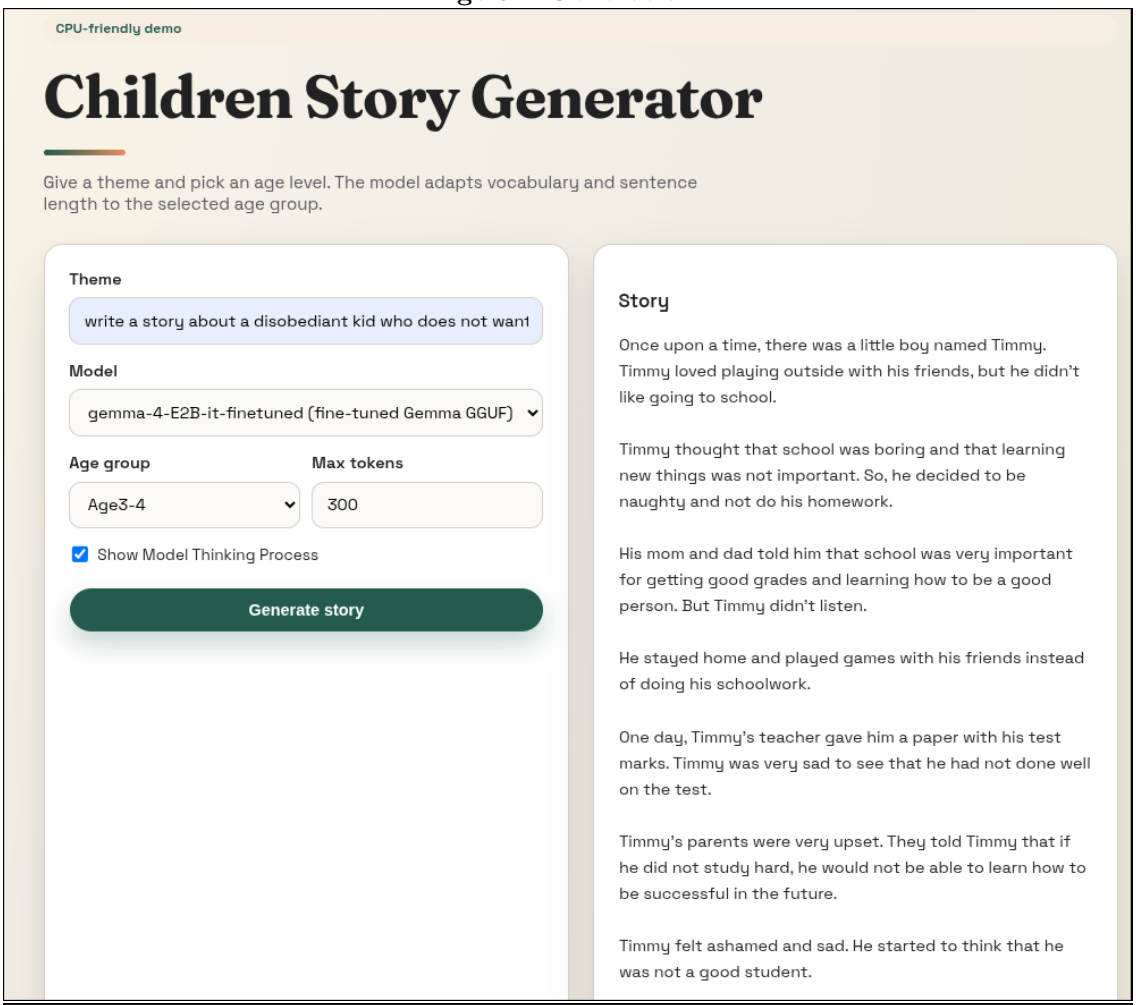
7. Evaluation and Results

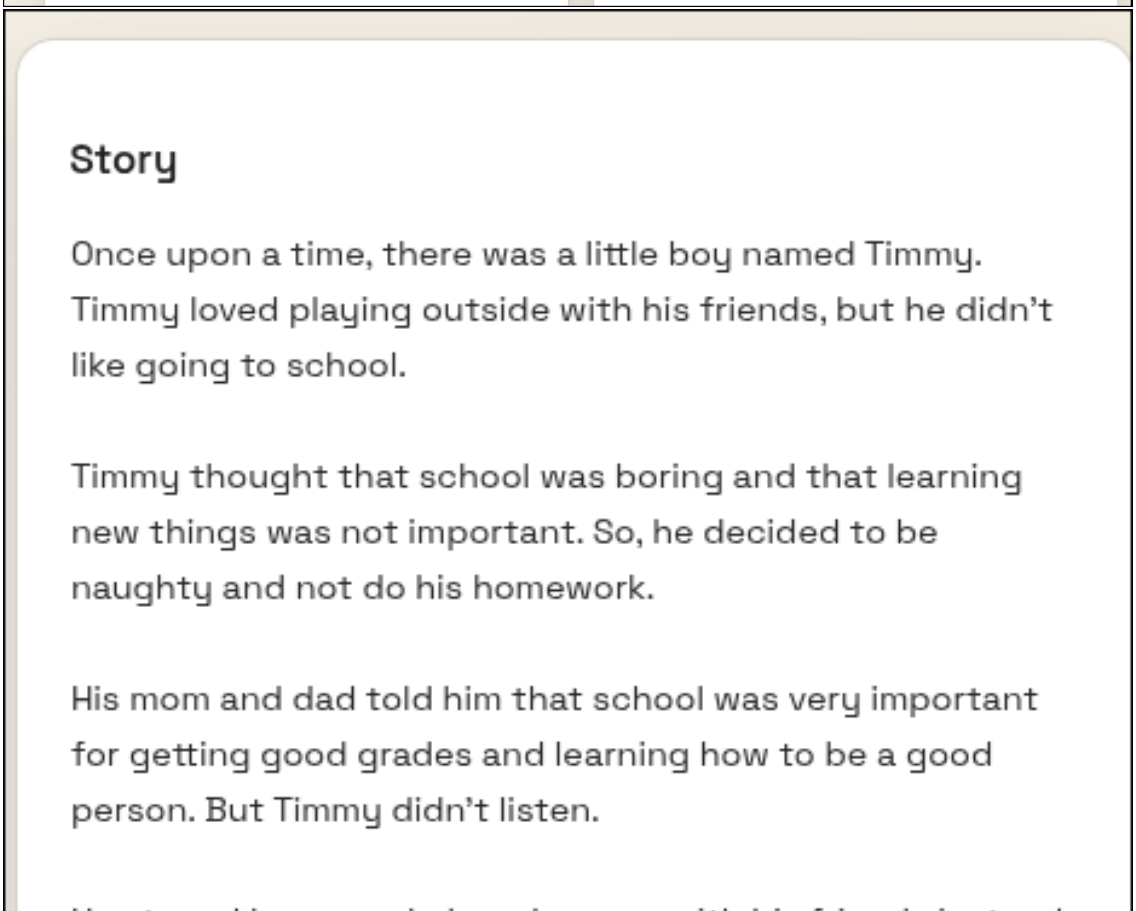
7.1. Qualitative Examples (Gemma 4 E2B-IT)

Post fine-tuning, the model reliably adhered to the age-level prefixes, adjusting its sentence complexity and vocabulary seamlessly.

Prompt: "Write a story about a disobedient kid who does want to study and ends up failing school."

Age 3-4 Generation:





As seen in the results, the model successfully uses more complex language and grammar for the older age group. It also adapts the tone: the ending in the 3-4 age group is more positive and simple, while the ending in the 11-12 age group is more realistic and complex. This demonstrates the model's ability to learn storytelling styles suited to the emotional and intellectual differences between age groups.

7.2. Comparison with Base Models

When compared to the base `gpt2-xl` and `gemma-4-E4B-it` models, the fine-tuned models (`gpt2-small` and `gemma-4-E2B-it`) showed significantly better adherence to the specific formatting and stylistic constraints of children's stories. The base models, while possessing stronger general reasoning, often ignored the age-specific constraints and generated generic or overly complex narratives.

8. Discussion and Limitations

While the fine-tuned models successfully generated formatted, age-appropriate children's stories, several challenges and limitations remain:

- **Vocabulary Size Overhead:** The 256k vocabulary size of Gemma 4 makes cross-entropy loss computation extremely expensive in VRAM, restricting training batch sizes and evaluation strategies.
- **Catastrophic Forgetting:** By fine-tuning purely on children's stories, the model's general instructional capability may degrade outside of this specific domain.
- **Hallucinations:** Without grounding constraints, the model may occasionally generate nonsensical plot resolutions. Future work could involve RAG (Retrieval-Augmented Generation) to ground the stories in specific educational facts.

9. Technologies Used

PyTorch & Transformers: PyTorch powers the underlying tensor math, while the Hugging Face Transformers library handles model loading, tokenization, and the training loop abstraction.

PEFT & BitsAndBytes: The PEFT library manages the LoRA adapter injection, while BitsAndBytes enables the NF4 precision loading essential for QLoRA on consumer hardware.

llama.cpp: Written in C/C++, it provides the engine for converting the PyTorch model into the highly optimized GGUF format for edge inference.

Kaggle: Kaggle's cloud environment, specifically the dual T4 GPU accelerators, served as the primary hardware infrastructure for the fine-tuning process.

FastAPI: Powers the local API microservice to serve the models for generation.

Web UI (HTML/JS/CSS): A lightweight frontend (served by FastAPI) to interact with the models dynamically.

Docker: Containerization to ensure environment consistency.

GitHub: Version control and repository hosting.

10. Local Reproduction Guide

To fully reproduce this project locally (e.g., on a personal CPU without GPU acceleration), follow these steps. **Note:** Downloading all models can require significant storage space. The web app has been designed to start gracefully and make all models optional; you only need to download the ones you wish to test.

10.1. 1. Environment Setup

Ensure you have an environment manager like `micromamba` or `conda` installed.

Install Dependencies

```
micromamba create -n NLP python=3.10
micromamba activate NLP
python -m pip install torch --index-url https://download.pytorch.org/whl/cpu
python -m pip install -r requirements.txt
```

10.2. 2. Download the Models (Optional)

You can download specific models you want to try. Run the `download_model.sh` script and specify the Hugging Face repository and local directory.

Example: Downloading Fine-Tuned GPT-2 Small

Download GPT-2 Small

```
export REPO_ID="khedim/NLP-MINI-PROJECT"
export MODEL_DIR="models/story-gpt2"
bash scripts/download_model.sh
```

Example: Downloading Fine-Tuned Gemma (GGUF)

Download Gemma E2B

```
export REPO_ID="khedim/NLP-MINI-PROJECT"
export MODEL_DIR="models/gemma-4-E2B-it-finetuned"
bash scripts/download_model.sh
```

10.3. 3. Run the API and Web UI

Start the local server using the provided script.

Run Local Server

```
bash scripts/run_api.sh
```

Once started, open http://localhost:8000/ in your web browser. The app will automatically detect any models present in the `models/` directory and make them available in the dropdown menu. If you only downloaded one model, the app will safely load it without crashing.